

QUBO Penalty Calibration

Maicol Nicolini

14 settembre 2026

Indice

1	Introduzione	2
2	Problem-based penalty calibration	3
2.1	Formulazione QUBO del WFLO	3
2.2	Principio generale	3
2.3	Vincolo sulla cardinalità	4
2.3.1	Stima conservativa sulla candidate grid	4
2.3.2	Calibrazione mediante una soluzione feasible di riferimento	5
2.4	Vincolo sulla distanza minima	6
2.4.1	Stima conservativa sulla candidate grid	6
2.4.2	Calibrazione mediante una soluzione feasible di riferimento	7
3	Relaxation-based penalty calibration	9
3.1	Il problema Big-M	9
3.2	Upper bound da una soluzione feasible	9
3.3	Lower bound della funzione obiettivo	10
3.4	Teoria della SDP relaxation	10
3.4.1	Lifting	10
3.4.2	Formulazione esatta	11
3.4.3	Relaxation	11
3.5	Perché la SDP fornisce un lower bound	12
3.6	Interpretazione della penalty SDP	12
3.7	Workflow relaxation-based	12
4	Confronto tra i due approcci	13
4.1	Problem-based	13
4.2	Relaxation-based	13
4.3	Strategia consigliata	14
5	Riferimenti	15

Capitolo 1

Introduzione

Consideriamo una formulazione QUBO del tipo

$$H(\mathbf{z}) = H_{\text{obj}}(\mathbf{z}) + \sum_r \lambda_r H_r(\mathbf{z}),$$

dove H_{obj} rappresenta la funzione obiettivo del problema originale e i termini H_r codificano i vincoli tramite penalità.

La scelta dei coefficienti λ_r è cruciale: una penalità troppo piccola può rendere conveniente la violazione del vincolo, mentre una penalità eccessivamente grande può peggiorare la scalatura numerica del QUBO e, nel caso di solver quantum o quantum-inspired, alterare sfavorevolmente la scala energetica del problema.

L'obiettivo di questo documento è raccogliere due famiglie di strategie per la calibrazione delle penalty:

1. **problem-based penalty calibration**, in cui si sfrutta direttamente la struttura del problema e l'interpretazione fisica dei termini della funzione obiettivo;
2. **relaxation-based penalty calibration**, in cui si costruisce un lower bound della funzione obiettivo mediante Semidefinite Programming (SDP) relaxation e si combina tale bound con una soluzione feasible nota.

Nel seguito utilizzeremo come esempio guida il problema di Wind Farm Layout Optimization (WFLO) formulato come QUBO.

Capitolo 2

Problem-based penalty calibration

2.1 Formulazione QUBO del WFLO

La formulazione considerata è

$$H(\mathbf{z}) = H_{\text{wake}}(\mathbf{z}) + \lambda_N \left(\sum_i z_i - K \right)^2 + \lambda_S \sum_{(i,j) \in \mathcal{E}_{\text{inv}}} z_i z_j,$$

dove

$$H_{\text{wake}}(\mathbf{z}) = \sum_{i < j} L_{ij} z_i z_j.$$

Il coefficiente $L_{ij} \geq 0$ rappresenta la perdita energetica pairwise dovuta all'interazione aerodinamica tra le turbine nelle candidate locations i e j .

Nel caso del benchmark considerato,

$$K = 81.$$

Si assume di utilizzare la formulazione QUBO ridotta, nella quale il termine lineare

$$-A_0 \sum_i z_i$$

è stato rimosso. Tale termine è infatti costante nello spazio delle soluzioni feasible, poiché ogni soluzione valida contiene esattamente $K = 81$ turbine.

2.2 Principio generale

L'idea utilizzata per dimensionare le penalty è rispondere, per ciascun vincolo, alla domanda:

Qual è il massimo beneficio che il modello potrebbe ottenere violando questo vincolo?

Una volta stimato tale beneficio, si sceglie una penalità sufficientemente grande da rendere la violazione energeticamente sconsigliata.

In generale, si desidera una penalty:

- sufficientemente grande da favorire o garantire la feasibility;
- sufficientemente piccola da non dominare inutilmente la funzione obiettivo.

2.3 Vincolo sulla cardinalità

Il problema impone

$$\sum_i z_i = K.$$

Poiché $L_{ij} \geq 0$, in assenza di questo vincolo il modello sarebbe naturalmente incentivato a selezionare meno turbine, riducendo il numero di interazioni aerodinamiche.

Il problema consiste quindi nello stimare quanto possa diminuire

$$H_{\text{wake}}(\mathbf{z})$$

rimuovendo una turbina.

2.3.1 Stima conservativa sulla candidate grid

Consideriamo una generica candidate location i .

Se la turbina i viene rimossa, scompaiono tutte le interazioni che la coinvolgono:

$$L_{ij} z_i z_j.$$

In un layout composto da K turbine, la turbina i può interagire con al massimo $K-1$ turbine. Definiamo quindi

$$\mathcal{J}_i^{K-1} = \text{Top}_{K-1} \{L_{ij} : j \neq i\}.$$

Nel nostro caso,

$$K-1 = 80.$$

Una stima conservativa del massimo beneficio ottenibile rimuovendo la turbina i è

$$m_i^{\text{grid}} = \sum_{j \in \mathcal{J}_i^{K-1}} L_{ij}.$$

Questa quantità assume che la turbina i sia contemporaneamente selezionata con le $K-1$ candidate locations che producono le maggiori wake losses.

Iterando il procedimento per ogni i si ottiene

$$m_{\max}^{\text{grid}} = \max_i m_i^{\text{grid}}.$$

Il termine di cardinalità è

$$H_N(\mathbf{z}) = \lambda_N \left(\sum_i z_i - K \right)^2.$$

Rimuovendo una sola turbina:

$$(K-1-K)^2 = 1,$$

quindi la penalità pagata è esattamente

$$\lambda_N.$$

Per rendere la rimozione non conveniente rispetto al massimo beneficio stimato, scegliamo

$$\lambda_N > m_{\max}^{\text{grid}}.$$

Introducendo un safety factor $\alpha_N > 1$:

$$\lambda_N = \alpha_N m_{\max}^{\text{grid}}$$

con, ad esempio,

$$\alpha_N \in [1.2, 1.8].$$

Questa stima è prudentiale poiché considera un worst-case costruito sull'intero spazio delle candidate locations.

2.3.2 Calibrazione mediante una soluzione feasible di riferimento

Se è disponibile un layout feasible di riferimento composto esattamente da K turbine, si può ottenere una stima meno conservativa.

Sia

$$\mathbf{z}^{\text{ref}} \in \{0, 1\}^N$$

una soluzione feasible tale che

$$\sum_i z_i^{\text{ref}} = K.$$

Definiamo

$$\mathcal{S}_{\text{ref}} = \left\{ i : z_i^{\text{ref}} = 1 \right\},$$

con

$$|\mathcal{S}_{\text{ref}}| = K.$$

Per ciascuna turbina $i \in \mathcal{S}_{\text{ref}}$, la wake loss marginale è

$$m_i^{\text{ref}} = \sum_{\substack{j \in \mathcal{S}_{\text{ref}} \\ j \neq i}} L_{ij}.$$

Equivalentemente,

$$m_i^{\text{ref}} = \sum_{j \neq i} L_{ij} z_j^{\text{ref}}.$$

La quantità m_i^{ref} rappresenta esattamente la riduzione della pairwise wake-loss objective che si otterrebbe rimuovendo la turbina i dal layout di riferimento.

Definiamo

$$m_{\max}^{\text{ref}} = \max_{i \in \mathcal{S}_{\text{ref}}} m_i^{\text{ref}}.$$

La cardinality penalty viene quindi calibrata come

$$\lambda_N = \alpha_N m_{\max}^{\text{ref}}$$

con $\alpha_N > 1$.

In generale ci aspettiamo

$$m_{\max}^{\text{ref}} \leq m_{\max}^{\text{grid}},$$

poiché la soluzione di riferimento contiene interazioni effettivamente presenti in un layout feasible, mentre la stima grid-based combina interazioni worst-case che potrebbero non essere simultaneamente realizzabili.

La soluzione feasible può essere, ad esempio:

- il baseline fornito dal benchmark;
- una soluzione ottenuta con OR-Tools;
- una soluzione prodotta da una greedy heuristic;
- un'altra soluzione feasible nota.

Questa seconda procedura è una calibrazione empirica e locale: fornisce una stima più rappresentativa di un layout realistico, ma non costituisce necessariamente un bound globale valido per ogni configurazione della candidate grid.

2.4 Vincolo sulla distanza minima

Definiamo l'insieme delle coppie non ammissibili come

$$\mathcal{E}_{\text{inv}} = \{(i, j) : \|p_i - p_j\| < d_{\min}\}.$$

Nel benchmark considerato,

$$d_{\min} = 396 \text{ m.}$$

Il termine di penalità è

$$H_S(\mathbf{z}) = \lambda_S \sum_{(i,j) \in \mathcal{E}_{\text{inv}}} z_i z_j.$$

Se una coppia non ammissibile viene selezionata, la funzione obiettivo riceve una penalità pari a λ_S .

La differenza rispetto alla cardinalità è che non è semplice associare il beneficio della violazione alla sola wake loss L_{ij} della coppia invalida.

Consentire una coppia non feasible può infatti modificare l'intera configurazione del layout e permettere al solver di ottenere una wake-loss objective complessivamente inferiore.

2.4.1 Stima conservativa sulla candidate grid

Un layout di K turbine contiene esattamente

$$\binom{K}{2}$$

interazioni pairwise.

Con $K = 81$:

$$\binom{81}{2} = 3240.$$

Definiamo

$$\mathcal{L}^{(K)}_{(2)} = \text{Top}_{(K)} \{L_{ij} : i < j\}.$$

Nel caso specifico:

$$\mathcal{L}^{3240} = \text{Top}_{3240} \{L_{ij} : i < j\}.$$

Definiamo quindi

$$E_{\max}^{\text{grid}} = \sum_{L_{ij} \in \mathcal{L}^{3240}} L_{ij}.$$

Questa quantità rappresenta un upper bound conservativo sulla wake-loss objective di qualunque configurazione di 81 turbine.

È un bound conservativo perché le 3240 interazioni selezionate globalmente potrebbero coinvolgere più di 81 candidate locations e quindi non essere simultaneamente realizzabili all'interno di una singola configurazione.

Per rendere una singola spacing violation più costosa di tale scala energetica si può scegliere

$$\lambda_S > E_{\max}^{\text{grid}}.$$

Con safety factor $\alpha_S > 1$:

$$\boxed{\lambda_S = \alpha_S E_{\max}^{\text{grid}}}$$

con, ad esempio,

$$\alpha_S \in [1.1, 1.5].$$

Questa strategia è robusta ma spesso estremamente conservativa.

2.4.2 Calibrazione mediante una soluzione feasible di riferimento

Sia $\mathbf{z}^{\text{ref}}$ una soluzione feasible contenente esattamente K turbine e rispettante il vincolo di distanza minima.

Definiamo la sua pairwise wake-loss objective:

$$E_{\text{ref}} = H_{\text{wake}}(\mathbf{z}^{\text{ref}}) = \sum_{i < j} L_{ij} z_i^{\text{ref}} z_j^{\text{ref}}.$$

Equivalentemente,

$$E_{\text{ref}} = \sum_{\substack{i < j \\ i, j \in S_{\text{ref}}}} L_{ij}.$$

Si sceglie la spacing penalty in modo che una singola violazione abbia un costo superiore all'intera wake-loss objective della soluzione feasible nota:

$$\lambda_S > E_{\text{ref}}.$$

Introducendo un safety factor $\alpha_S > 1$:

$$\boxed{\lambda_S = \alpha_S E_{\text{ref}}}$$

con, ad esempio,

$$\alpha_S = 1.2.$$

Infatti, per una soluzione infeasible $\mathbf{z}^{\text{inf}}$ con almeno una spacing violation,

$$H_S(\mathbf{z}^{\text{inf}}) \geq \lambda_S.$$

Poiché

$$H_{\text{wake}}(\mathbf{z}^{\text{inf}}) \geq 0,$$

segue che

$$H(\mathbf{z}^{\text{inf}}) \geq \lambda_S.$$

Se

$$\lambda_S > E_{\text{ref}},$$

allora

$$H(\mathbf{z}^{\text{inf}}) > E_{\text{ref}} = H_{\text{wake}}(\mathbf{z}^{\text{ref}}).$$

Quindi una configurazione contenente almeno una spacing violation non può battere la soluzione feasible di riferimento.

Anche in questo caso il valore ottenuto può essere sottoposto a sensitivity analysis, riducendo progressivamente λ_S e verificando empiricamente la feasibility delle soluzioni prodotte.

Capitolo 3

Relaxation-based penalty calibration

3.1 Il problema Big-M

Consideriamo un problema binario quadratico vincolato:

$$\min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x}$$

soggetto a

$$A\mathbf{x} = \mathbf{b}.$$

Per trasformarlo in un QUBO, il vincolo viene promosso a termine di penalità:

$$H_M(\mathbf{x}) = f(\mathbf{x}) + M\|A\mathbf{x} - \mathbf{b}\|^2.$$

Per una soluzione feasible la penalty è nulla. Per una soluzione infeasible è positiva.

Il valore M deve quindi essere sufficientemente grande affinché nessuna soluzione infeasible possa avere energia inferiore alla soluzione ottima feasible.

Una scelta eccessivamente grande di M introduce però il cosiddetto *Big-M problem*: la funzione obiettivo viene dominata dalla penalty, peggiorando la scala numerica dei coefficienti e, nel contesto quantum e quantum-inspired, la scala energetica dell'Hamiltoniana. L'obiettivo è quindi scegliere la più piccola penalty che renda la riformulazione esatta.

3.2 Upper bound da una soluzione feasible

Sia

$$\mathbf{x}_{\text{feas}}$$

una soluzione feasible nota. Il suo valore

$$U = f(\mathbf{x}_{\text{feas}})$$

costituisce un upper bound sul valore ottimo del problema vincolato:

$$f(\mathbf{x}^*) \leq U.$$

La soluzione feasible può provenire da:

- un solver classico con time limit limitato;
- una greedy heuristic;

- una soluzione benchmark;
- una soluzione precedentemente calcolata.

Più la soluzione feasible è buona, più piccolo sarà U e più stretta sarà la penalty ottenuta.

3.3 Lower bound della funzione obiettivo

Rimuoviamo temporaneamente i vincoli e consideriamo

$$\min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}).$$

Supponiamo di conoscere un lower bound L tale che

$$L \leq \min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}).$$

La differenza

$$U - L$$

rappresenta una stima conservativa del massimo vantaggio che una soluzione potrebbe ottenere ignorando i vincoli.

Per rendere sconveniente tale vantaggio è sufficiente scegliere

$$M > U - L.$$

Introducendo un margine $\delta > 0$:

$$\boxed{M = U - L + \delta}$$

Il lower bound viene trovata tramite una procedura di *relaxation*. Ne esistono diverse, le più importanti sono due: *linear relaxation* e *Semidefinite Programming relaxation*, ci concentriamo su quest'ultima.

$$\boxed{M_{\text{SDP}} = f(\mathbf{x}_{\text{feas}}) - f_{\text{SDP}}^* + \delta}$$

3.4 Teoria della SDP relaxation

Consideriamo una funzione binaria quadratica generale:

$$f(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x} + \mathbf{L}^T \mathbf{x}, \quad \mathbf{x} \in \{0,1\}^n.$$

L'obiettivo è ottenere un lower bound senza risolvere esattamente il problema combinatorio.

3.4.1 Lifting

Definiamo il vettore aumentato

$$\mathbf{y} = \begin{pmatrix} 1 \\ \mathbf{x} \end{pmatrix}$$

e la matrice lifted

$$Y = \mathbf{y}\mathbf{y}^T = \begin{pmatrix} 1 & \mathbf{x}^T \\ \mathbf{x} & \mathbf{x}\mathbf{x}^T \end{pmatrix}.$$

La parte in basso a destra contiene direttamente i prodotti pairwise:

$$Y_{i+1,j+1} = x_i x_j.$$

Definendo

$$\tilde{Q} = \begin{pmatrix} 0 & \frac{1}{2}\mathbf{L}^T \\ \frac{1}{2}\mathbf{L} & Q \end{pmatrix},$$

la funzione obiettivo può essere riscritta come

$$f(\mathbf{x}) = \text{Tr} \left(Y^T \tilde{Q} \right).$$

Dopo il lifting, la funzione obiettivo è quindi lineare nella matrice Y .

3.4.2 Formulazione esatta

Se

$$Y = \mathbf{y}\mathbf{y}^T,$$

allora Y soddisfa due proprietà:

$$Y \succeq 0$$

e

$$\text{rank}(Y) = 1.$$

La condizione

$$Y \succeq 0$$

significa che Y è positiva semidefinita, cioè

$$\mathbf{v}^T Y \mathbf{v} \geq 0$$

per qualsiasi vettore $\mathbf{v}$.

Infatti,

$$\mathbf{v}^T Y \mathbf{v} = \mathbf{v}^T \mathbf{y}\mathbf{y}^T \mathbf{v} = (\mathbf{y}^T \mathbf{v})^2 \geq 0.$$

Il vincolo problematico è

$$\text{rank}(Y) = 1,$$

che rende il problema non convesso.

3.4.3 Relaxation

La SDP relaxation rimuove il vincolo di rango 1 e mantiene la condizione semidefinita positiva. Inoltre, poiché per una variabile binaria vale

$$x_i^2 = x_i,$$

si impone la relazione lineare

$$Y_{1,i+1} = Y_{i+1,i+1}.$$

Si aggiungono inoltre bounds del tipo

$$0 \leq Y_{ij} \leq 1.$$

La relaxation assume quindi la forma

$$\min_Y \text{Tr} \left(Y^T \tilde{Q} \right)$$

soggetta a

$$Y \succeq 0,$$

$$Y_{1,i+1} = Y_{i+1,i+1},$$

$$0 \leq Y_{ij} \leq 1.$$

3.5 Perché la SDP fornisce un lower bound

Nel problema originale la matrice Y deve necessariamente essere generata da un vettore binario e avere rango 1. Nella SDP ammettiamo anche matrici positive semidefinite che non corrispondono ad alcuna soluzione binaria reale.

Abbiamo quindi

$$\mathcal{F}_{\text{binary}} \subseteq \mathcal{F}_{\text{SDP}}.$$

Poiché stiamo minimizzando,

$$f_{\text{SDP}}^* \leq f_{\text{binary}}^*$$

e il valore ottimo della SDP è quindi un lower bound valido per il problema binario originale.

3.6 Interpretazione della penalty SDP

Una volta disponibili:

- una soluzione feasible con costo

$$U = f(\mathbf{x}_{\text{feas}});$$

- un lower bound

$$f_{\text{SDP}}^*;$$

si definisce

$$M_{\text{SDP}} = U - f_{\text{SDP}}^* + \delta.$$

L'interpretazione è:

$$\underbrace{f_{\text{SDP}}^*}_{\text{scenario ottimistico}} \longrightarrow \underbrace{f(\mathbf{x}_{\text{feas}})}_{\text{soluzione feasible nota}}.$$

La differenza tra questi due valori rappresenta una stima conservativa del massimo vantaggio che una soluzione potrebbe ottenere ignorando i vincoli.

La SDP viene quindi utilizzata come preprocessing classico per ottenere una penalty più stretta, non come solver finale del QUBO.

3.7 Workflow relaxation-based

Il procedimento può essere riassunto come segue:

1. trovare una soluzione feasible $\mathbf{x}_{\text{feas}}$;

2. calcolare

$$U = f(\mathbf{x}_{\text{feas}});$$

3. rimuovere temporaneamente i vincoli;

4. risolvere la SDP relaxation;

5. ottenere

$$f_{\text{SDP}}^*;$$

6. scegliere

$$M_{\text{SDP}} = U - f_{\text{SDP}}^* + \delta;$$

7. costruire il QUBO finale con la penalty ottenuta;

8. validare la feasibility con il solver target.

Capitolo 4

Confronto tra i due approcci

Le due strategie rispondono allo stesso obiettivo: ottenere penalty sufficientemente grandi da rendere sconveniente la violazione dei vincoli, evitando però valori inutilmente elevati.

4.1 Problem-based

L'approccio problem-based sfrutta direttamente:

- la struttura della funzione obiettivo;
- l'interpretazione fisica dei termini;
- informazioni specifiche sul vincolo;
- eventuali soluzioni feasible note.

Vantaggi:

- semplice da interpretare;
- spesso economico computazionalmente;
- può sfruttare fortemente la struttura specifica del problema.

Limiti:

- le formule dipendono dal problema;
- i bound conservativi possono essere molto grandi;
- le versioni basate su una reference solution sono in genere empiriche e locali.

4.2 Relaxation-based

L'approccio SDP costruisce invece una ricetta più generale:

$$M_{\text{SDP}} = U - L + \delta.$$

Vantaggi:

- formulazione generica;
- lower bound matematicamente motivato;

- spesso produce valori Big-M più stretti rispetto a bound elementari.

Limiti:

- richiede la soluzione di una SDP;
- la SDP densa può diventare costosa per problemi di grandi dimensioni;
- la qualità della penalty dipende dalla qualità sia dell'upper bound feasible sia del lower bound SDP.

4.3 Strategia consigliata

Una strategia pratica può combinare i due approcci:

problem-based bound $\rightarrow$ SDP tightening $\rightarrow$ sensitivity analysis $\rightarrow$ solver target

Il bound problem-based fornisce una prima scala interpretabile. La SDP relaxation consente poi di restringere la penalty in modo più sistematico. Infine, una sensitivity analysis permette di verificare empiricamente che la penalty scelta rimanga sufficientemente robusta sul solver target.

Capitolo 5

Riferimenti

E. Alessandroni, S. Ramos-Calderer, I. Roth, E. Traversi e L. Aolita, *Alleviating the quantum Big-M problem*, arXiv:2307.10379v4, 2025.